



CSE-2422

Computer Algorithms Lab

**Department of Computer Science and
Engineering
International Islamic University Chittagong**

CSE- 2422 Computer Algorithms Lab Syllabus

Prerequisite: CSE-2322 (Data Structures Lab)

Co-requisite: CSE-2421 (Computer Algorithms)

Credit Hours: 1.5

Course Description:

This course introduces students to the general tools and techniques for analyzing and designing computer algorithms. Initially necessary mathematical preliminaries required for analyzing and designing computer algorithms are taught. Then this course familiarizes students with several algorithmic approaches and corresponding problems. To take this course, students should be able to program in a standard programming language preferably in C/C++. They should have familiarity with basic data structures. In this course, students are expected to implement and apply the algorithms and techniques taught in CSE-2421. Some mathematical maturity, habit of critical thinking and enthusiasm for solving problems are expected.

Course Objectives:

The objectives of this course are to 1. familiarize with the major algorithms and data structures 2. proficiently apply important algorithmic design paradigms and methods of analysis 3. synthesize efficient algorithms in common engineering design situations

Course Learning Outcomes:

Upon successful completion of this course, students will be able to:

1. Implement and debug algorithms using programming languages and use algorithmic libraries and industry-standard tools for faster problem-solving and performance evaluation.
2. Solve algorithmic problems under real-world constraints using Competitive Programming platforms, optimizing code for efficiency and correctness while analyzing performance trade-offs.

Course Plan:

	Content	Duration	CLOs
1.	Sorting Algorithms: Insertion Sort Merge Sort, Quick Sort (with Randomization)	2 weeks	CLO1-CLO3
2.	Sorting Algorithms: Heap and Heap Sort Priority Queue using Binary Heap, Counting Sort Radix Sort	2 weeks	CLO1-CLO3
3.	Dynamic Programming & Optimization: Matrix Chain Multiplication Longest Common Subsequence (LCS), Solving problems with the technique of Memoization Competitive programming problem with dynamic programming	2 week	CLO1-CLO3
4.	Greedy Algorithms & Huffman Coding Activity Selection Problem Huffman Tree & Prefix Generation	2 weeks	CLO1-CLO3
5.	Graph Algorithms Breadth First Search (BFS) Depth First Search (DFS) Kruskal's Algorithm (Minimum Spanning Tree) Prim's Algorithm (Minimum Spanning Tree) Dijkstra's Algorithm (Single-Source Shortest Path) Bellman-Ford's Algorithm (Single-Source Shortest Path)	3 weeks	CLO1-CLO3
6.	Advanced Graph Algorithms : Floyd-Warshall's Algorithm (All-Pairs Shortest Path), Computational Geometry, Graham Scan Algorithm	2 weeks	CLO1-CLO3
7.	String Matching Algorithm Naive String Matching Algorithm Rabin-Karp Algorithm	1 week	CLO1-CLO3
8.	Miscellaneous Algorithms & Problem Solving Extended Euclid's Algorithm (GCD) Prime Number Generation Algorithms N-Queen Problem Backtracking Problems Modular Exponentiation Fractional Knapsack Problem	1 week	CLO1-CLO3

1. Sorting Algorithms:

AIM: To understand the fundamentals of different sorting algorithms.

Introduction:

Sorting algorithms are methods used to arrange elements of a list or array in a certain order—typically **ascending** or **descending**. These algorithms are foundational in computer science and are used to organize data to make searching, processing, and analyzing more efficient. In this module, we will learn about Insertion Sort, Merge Sort, Quick Sort.

Lab Questions:

1. Implement Insertion Sort.

Code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void display(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {5, 2, 4, 6, 1, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    insertionSort(arr, n);
    cout << "Insertion Sort: ";
    display(arr, n);
}
```

```
    return 0;
}
```

Output:

```
Insertion sort: 1 2 3 4 5 6
```

2. Implement Merge Sort.

Code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int i = 0; i < n2; i++) R[i] = arr[m + 1 + i];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void display(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);
    mergeSort(arr, 0, n - 1);
    cout << "Merge Sort: ";
    display(arr, n);
    return 0;
}
```

}

Output:

Merge Sort: 3 9 10 27 38 43 82

3. Implement Quick Sort.

Code:

```

#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot)
            swap(arr[++i], arr[j]);
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

void display(int arr[], int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    quickSort(arr, 0, n - 1);
    cout << "Quick Sort: ";
    display(arr, n);
    return 0;
}

```

Output:

```
Quick Sort: 1 5 7 8 9 10
```

2. Sorting Algorithms:

Aim: To implement efficient sorting algorithms including Heap Sort, Counting Sort, and Radix Sort.

Introduction:

Sorting is a fundamental operation in computer science that arranges data in a particular order, typically ascending or descending. Efficient sorting algorithms are crucial for optimizing the performance of other algorithms, especially in searching and data processing. This lab explores four powerful sorting techniques: Heap Sort, Priority Queue using Binary Heap, Counting Sort, and Radix Sort.

- **Heap Sort** is a comparison-based sorting technique based on a binary heap data structure.
- **Priority Queue using Binary Heap** allows elements to be inserted with priorities and enables efficient retrieval of the highest (or lowest) priority element.
- **Counting Sort** is a non-comparison sorting algorithm that works efficiently for small ranges of integers.
- **Radix Sort** processes integer keys by individual digits which share the same position and value.

Lab Questions:

1. Implement Heap Sort:

Code:

```
#include <iostream>
using namespace std;

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
```



```

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main() {
    int arr[] = {4, 10, 3, 5, 1};
    int n = sizeof(arr)/sizeof(arr[0]);

    cout << "Original: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    heapSort(arr, n);

    cout << "Sorted: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}

```

Output:

```

Heap Sort:
Original: 4 10 3 5 1
Sorted: 1 3 4 5 10

```

2. Implement Priority Queue using Binary Heap

Code:

```

#include <iostream>
#include <vector>
using namespace std;

class PriorityQueue {
    vector<int> heap;

    void heapifyDown(int i) {
        int smallest = i;
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < heap.size() && heap[left] < heap[smallest])
            smallest = left;
        if (right < heap.size() && heap[right] < heap[smallest])
            smallest = right;

        if (smallest != i) {
            swap(heap[i], heap[smallest]);
            heapifyDown(smallest);
        }
    }

public:
    void insert(int val) {
        heap.push_back(val);
        int i = heap.size() - 1;
        while (i > 0 && heap[i] < heap[(i - 1) / 2]) {
            swap(heap[i], heap[(i - 1) / 2]);
            i = (i - 1) / 2;
        }
    }

    void deleteMin() {
        if (heap.empty()) return;
        heap[0] = heap.back();
        heap.pop_back();
        heapifyDown(0);
    }

    void display() {
        for (int val : heap) cout << val << " ";
        cout << endl;
    }
};

int main() {
    PriorityQueue pq;
    pq.insert(5);

```

```

    pq.insert(3);
    pq.insert(8);
    pq.insert(1);

    cout << "Priority Queue after insertions: ";
    pq.display();

    pq.deleteMin();
    cout << "After deleting minimum: ";
    pq.display();

    return 0;
}

```

Output:

```

Priority Queue:
Priority Queue after insertions: 1 3 8 5
After deleting minimum: 3 5 8

```

3. Implement Counting Sort**Code:**

```

#include <iostream>
#include <algorithm>
using namespace std;

void countingSort(int arr[], int n) {
    int maxVal = *max_element(arr, arr + n);
    int* count = new int[maxVal + 1]{};

    for (int i = 0; i < n; i++)
        count[arr[i]]++;

    int index = 0;
    for (int i = 0; i <= maxVal; i++) {
        while (count[i]-- > 0)
            arr[index++] = i;
    }

    delete[] count;
}

```

```

int main() {
    int arr[] = {4, 2, 2, 8, 3};
    int n = sizeof(arr)/sizeof(arr[0]);

    cout << "Original: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    countingSort(arr, n);

    cout << "Sorted: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

Output:

```

Counting Sort:
Original: 4 2 2 8 3
Sorted: 2 2 3 4 8

```

4. Implement Radix Sort

Code:

```

#include <iostream>
using namespace std;

int getMax(int arr[], int n) {
    int mx = arr[0];
    for (int i = 1; i < n; i++)
        if (arr[i] > mx)
            mx = arr[i];
    return mx;
}

void countingSortForRadix(int arr[], int n, int exp) {
    int output[n];
    int count[10] = {0};
}

```

```

    for (int i = 0; i < n; i++)
        count[(arr[i] / exp) % 10]++;

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for (int i = n - 1; i >= 0; i--) {
        output[count[(arr[i] / exp) % 10] - 1] = arr[i];
        count[(arr[i] / exp) % 10]--;
    }

    for (int i = 0; i < n; i++)
        arr[i] = output[i];
}

void radixSort(int arr[], int n) {
    int m = getMax(arr, n);
    for (int exp = 1; m / exp > 0; exp *= 10)
        countingSortForRadix(arr, n, exp);
}

int main() {
    int arr[] = {170, 45, 75, 90, 802, 24, 2, 66};
    int n = sizeof(arr)/sizeof(arr[0]);

    cout << "Original: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    radixSort(arr, n);

    cout << "Sorted: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;

    return 0;
}

```

3. Dynamic Programming & Optimization:

Aim: Thoroughly understand Dynamic Programming & Optimization: Matrix Chain Multiplication, Longest Common Subsequence(LCS) , Memoization, and Competitive Programming.

Introduction:

Dynamic Programming (DP) is a method for solving complex problems by breaking them down into simpler subproblems. It is applicable when the problem has overlapping subproblems and optimal substructure. This lab covers the Matrix Chain Multiplication problem, the Longest Common Subsequence (LCS) problem, and the concept of Memoization, including a practical problem from competitive programming.

Lab Questions:

1. Implement Matrix Chain Multiplication using Dynamic Programming

Code:

```
#include <iostream>
#include <climits>
using namespace std;

int matrixChainOrder(int p[], int n) {
    int m[n][n];
    for (int i = 1; i < n; i++)
        m[i][i] = 0;

    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j])
                    m[i][j] = q;
            }
        }
    }
}
```

```

        return m[1][n - 1];
    }

int main() {
    int arr[] = {1, 2, 3, 4};
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Minimum number of multiplications: " << matrixChainOrder(arr, n);
    return 0;
}

```

Output:

```

Matrix Chain Multiplication:
Minimum number of multiplications: 18

```

2. Find the Longest Common Subsequence among two given strings.

Code:

```

#include <iostream>
#include <string>
#include <vector>
using namespace std;

int lcs(string X, string Y) {
    int m = X.length();
    int n = Y.length();
    vector<vector<int>> L(m + 1, vector<int>(n + 1));

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i - 1] == Y[j - 1])
                L[i][j] = L[i - 1][j - 1] + 1;
            else
                L[i][j] = max(L[i - 1][j], L[i][j - 1]);
        }
    }

    return L[m][n];
}

int main() {

```

```

string X = "AGGTAB";
string Y = "GXTXAYB";
cout << "Length of LCS: " << lcs(X, Y);
return 0;
}

```

Output:

```

Longest Common Subsequence:
Length of LCS: 4

```

3. Find fibonacci series using recursion and memoization.

Code:

```

#include<bits/stdc++.h>
using namespace std;

const int N = 55;
int f[N];
bool is_computed[N];
int fibo(int i) {
    if (i == 0) return 0;
    if (i == 1) return 1;
    if (is_computed[i]) return f[i];
    f[i] = fibo(i - 1) + fibo(i - 2);
    is_computed[i] = true;
    return f[i];
}

int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    cout << fibo(10) << '\n';
    return 0;
}

```

Output:

```

55

```


4. Greedy Algorithms

Aim: To understand and solve Greedy Algorithms & Huffman Coding: Activity Selection Problem, Huffman Tree & Prefix Generation.

Introduction:

Greedy algorithms build up a solution piece by piece, always choosing the next piece that offers the most immediate benefit. This lab focuses on implementing the Activity Selection Problem and Huffman Coding, two classic examples of greedy algorithms. Huffman Coding is especially useful in data compression, while Activity Selection models real-world scheduling problems.

Lab Questions:

1. Implement the Activity Selection Problem using a greedy approach.

Code:

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Activity {
    int start, finish;
};

bool compare(Activity a, Activity b) {
    return a.finish < b.finish;
}

void activitySelection(vector<Activity>& activities) {
    sort(activities.begin(), activities.end(), compare);
    cout << "Selected activities:\n";
    int lastFinish = activities[0].finish;
    cout << "(" << activities[0].start << ", " << activities[0].finish << ")\n";

    for (int i = 1; i < activities.size(); i++) {
        if (activities[i].start >= lastFinish) {
```

```

        cout << "(" << activities[i].start << ", " << activities[i].finish <<
    "\n";
        lastFinish = activities[i].finish;
    }
}

int main() {
    vector<Activity> activities = {{5, 9}, {1, 2}, {3, 4}, {0, 6}, {8, 9}, {5, 7}};
    activitySelection(activities);
    return 0;
}

```

Output:

Selected activities:

(1, 2)
(3, 4)
(5, 7)
(8, 9)

2. Implement the Huffman Coding Algorithm.**Code:**

```

#include <iostream>
#include <queue>
#include <unordered_map>
using namespace std;

struct Node {
    char ch;
    int freq;
    Node *left, *right;

    Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
};

struct compare {
    bool operator()(Node* l, Node* r) {
        return l->freq > r->freq;
    }
};

```

```

void printCodes(Node* root, string code) {
    if (!root) return;
    if (!root->left && !root->right)
        cout << root->ch << ": " << code << endl;
    printCodes(root->left, code + "0");
    printCodes(root->right, code + "1");
}

void buildHuffmanTree(unordered_map<char, int>& freqMap) {
    priority_queue<Node*, vector<Node*>, compare> pq;

    for (auto pair : freqMap)
        pq.push(new Node(pair.first, pair.second));

    while (pq.size() > 1) {
        Node* left = pq.top(); pq.pop();
        Node* right = pq.top(); pq.pop();

        Node* merged = new Node('\0', left->freq + right->freq);
        merged->left = left;
        merged->right = right;

        pq.push(merged);
    }

    Node* root = pq.top();
    printCodes(root, "");
}

int main() {
    unordered_map<char, int> freqMap = {
        {'a', 5}, {'b', 9}, {'c', 12}, {'d', 13}, {'e', 16}, {'f', 45}
    };

    cout << "Huffman Codes:\n";
    buildHuffmanTree(freqMap);
    return 0;
}

```

Output:

```

Huffman Codes:
a: 1100
b: 1101
c: 100
d: 101

```

e: 111
f: 0

5. Graph Algorithms

Aim: Implementation of Graph Algorithms: BFS, DFS, Kruskal's, Prim's, Dijkstra's, Bellman-Ford's.

Introduction:

Graph algorithms are crucial in computer science for solving problems like navigation, connectivity, shortest paths, and network designs. This lab explores fundamental graph traversal techniques and optimization algorithms: BFS, DFS, Kruskal's and Prim's MST algorithms, and shortest path algorithms such as Dijkstra's and Bellman-Ford's.

Lab Questions:

1. Implement Breadth-First Search (BFS) on graph.

Code:

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void BFS(int start, vector<vector<int>>& adj, int V) {
    vector<bool> visited(V, false);
    queue<int> q;
    q.push(start);
    visited[start] = true;
    cout << "BFS: ";
    while (!q.empty()) {
        int node = q.front(); q.pop();
        cout << node << " ";
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

```

    cout << endl;
}

int main() {
    int V = 5;
    vector<vector<int>> adj(V);
    adj[0] = {1, 2};
    adj[1] = {0, 3};
    adj[2] = {0, 3};
    adj[3] = {1, 2, 4};
    adj[4] = {3};

    BFS(0, adj, V);

    return 0;
}

```

Output:

```
BFS: 0 1 2 3 4
```

2. Implement Depth-First Search(DFS) on graph.

Code:

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

void BFS(int start, vector<vector<int>>& adj, int V) {
    vector<bool> visited(V, false);
    queue<int> q;
    q.push(start);
    visited[start] = true;
    cout << "BFS: ";
    while (!q.empty()) {
        int node = q.front(); q.pop();
        cout << node << " ";
        for (int neighbor : adj[node]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
            }
        }
    }
}

```

```

        q.push(neighbor);
    }
}
cout << endl;
}

void DFSUtil(int node, vector<vector<int>>& adj, vector<bool>& visited) {
    visited[node] = true;
    cout << node << " ";
    for (int neighbor : adj[node]) {
        if (!visited[neighbor])
            DFSUtil(neighbor, adj, visited);
    }
}

void DFS(int start, vector<vector<int>>& adj, int V) {
    vector<bool> visited(V, false);
    cout << "DFS: ";
    DFSUtil(start, adj, visited);
    cout << endl;
}

int main() {
    int V = 5;
    vector<vector<int>> adj(V);
    adj[0] = {1, 2};
    adj[1] = {0, 3};
    adj[2] = {0, 3};
    adj[3] = {1, 2, 4};
    adj[4] = {3};

    BFS(0, adj, V);
    DFS(0, adj, V);
    return 0;
}

```

Output:

```
DFS: 0 1 3 2 4
```

3. Implement Kruskal's Algorithm to find the Minimum Spanning Tree.

Code:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Edge {
    int u, v, weight;
};

bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
}

int find(int parent[], int i) {
    if (parent[i] != i)
        parent[i] = find(parent, parent[i]);
    return parent[i];
}

void unite(int parent[], int rank[], int x, int y) {
    int xroot = find(parent, x);
    int yroot = find(parent, y);
    if (rank[xroot] < rank[yroot])
        parent[xroot] = yroot;
    else if (rank[xroot] > rank[yroot])
        parent[yroot] = xroot;
    else {
        parent[yroot] = xroot;
        rank[xroot]++;
    }
}

void Kruskal(int V, vector<Edge>& edges) {
    sort(edges.begin(), edges.end(), compare);
    int parent[V], rank[V] = {};
    for (int i = 0; i < V; i++) parent[i] = i;

    cout << "Edges in MST (Kruskal):\n";
    for (Edge e : edges) {
        int x = find(parent, e.u);
        int y = find(parent, e.v);
        if (x != y) {
            cout << e.u << " - " << e.v << " : " << e.weight << endl;
            unite(parent, rank, x, y);
        }
    }
}

int main() {
    int V = 4;
    vector<Edge> edges = {

```



```

        {0, 1, 10}, {0, 2, 6}, {0, 3, 5}, {1, 3, 15}, {2, 3, 4}
    };
    Kruskal(V, edges);
    return 0;
}

```

Output:

```

Edges in MST (Kruskal):
2 - 3 : 4
0 - 3 : 5
0 - 2 : 6
0 - 1 : 10

```

4. Implement Prim's Algorithm to find the Minimum Spanning Tree

Code:

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int INF = 1e9;

typedef pair<int, int> pii; // {weight, node}

void Prim(int V, vector<vector<pii>>& adj) {
    vector<int> key(V, INF);
    vector<bool> inMST(V, false);
    priority_queue<pii, vector<pii>, greater<pii>> pq;

    key[0] = 0;
    pq.push({0, 0});

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        if (inMST[u]) continue;
        inMST[u] = true;
    }
}

```

```

        for (auto [v, w] : adj[u]) {
            if (!inMST[v] && w < key[v]) {
                key[v] = w;
                pq.push({w, v});
            }
        }
    }

    cout << "Key values (Prim):\n";
    for (int i = 0; i < V; i++) cout << i << " : " << key[i] << endl;
}

int main() {
    int V = 4;
    vector<vector<pii>> adj(V);
    adj[0].push_back({1, 10}); adj[0].push_back({2, 6}); adj[0].push_back({3, 5});
    adj[1].push_back({3, 15});
    adj[2].push_back({3, 4});

    Prim(V, adj);
    return 0;
}

```

Output:

```

Key values (Prim):
0 : 0
1 : 10
2 : 4
3 : 5

```

5. Implement Dijkstra's Algorithm for Single-Source Shortest Path finding.

Code:

```

#include <iostream>
#include <vector>
#include <queue>
using namespace std;

const int INF = 1e9;

typedef pair<int, int> pii; // {distance, vertex}

```

```

void Dijkstra(int V, vector<vector<pii>>& adj, int src) {
    vector<int> dist(V, INF);
    dist[src] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq;
    pq.push({0, src});

    while (!pq.empty()) {
        int u = pq.top().second;
        pq.pop();

        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }

    cout << "Distances from source " << src << ":\n";
    for (int i = 0; i < V; i++)
        cout << i << " : " << dist[i] << endl;
}

int main() {
    int V = 5;
    vector<vector<pii>> adj(V);
    adj[0] = {{1, 10}, {3, 5}};
    adj[1] = {{2, 1}, {3, 2}};
    adj[2] = {{4, 4}};
    adj[3] = {{1, 3}, {2, 9}, {4, 2}};
    adj[4] = {{0, 7}, {2, 6}};

    Dijkstra(V, adj, 0);
    return 0;
}

```

Output:

```

Distances from source 0:
0 : 0
1 : 10
2 : 6
3 : 5
4 : 9

```

6. Implement Bellman-Ford's Algorithm for Single-Source Shortest Path

Code:

```

#include <iostream>
#include <vector>
#include <climits>
using namespace std;

struct Edge {
    int u, v, weight;
};

void BellmanFord(int V, vector<Edge>& edges, int src) {

    vector<int> dist(V, INT_MAX);
    dist[src] = 0;

    for (int i = 0; i < V - 1; i++) {
        for (Edge e : edges) {
            if (dist[e.u] != INT_MAX && dist[e.u] + e.weight < dist[e.v]) {
                dist[e.v] = dist[e.u] + e.weight;
            }
        }
    }

    for (Edge e : edges) {
        if (dist[e.u] != INT_MAX && dist[e.u] + e.weight < dist[e.v]) {
            cout << "Negative weight cycle detected\n";
            return;
        }
    }

    cout << "Distances from source " << src << ":\n";
    for (int i = 0; i < V; i++) {
        if (dist[i] == INT_MAX)
            cout << i << " : INF\n"; // If no path exists
        else
            cout << i << " : " << dist[i] << endl;
    }
}

int main() {
    int V = 5; // Number of vertices
    vector<Edge> edges = {
        {0, 1, -1}, {0, 2, 4}, {1, 2, 3}, {1, 3, 2}, {1, 4, 2},
        {3, 2, 5}, {3, 1, 1}, {4, 3, -3}
    };
}

```

```
};  
  
BellmanFord(V, edges, 0);  
  
return 0;  
}
```

Output:

```
Distances from source 0:  
0 : 0  
1 : -1  
2 : 2  
3 : 1  
4 : 1
```

6. Advanced Graph Algorithms:

Aim: Understand and implement Floyd-Warshall's Algorithm for all-pairs shortest path, Graham Scan Algorithm for finding the convex hull of a set of points in 2D space.

Introduction:

The **Floyd-Warshall algorithm** is an all-pairs shortest path algorithm that finds the shortest path between every pair of vertices in a weighted graph. It works for both directed and undirected graphs and can handle graphs with negative weights (but not negative weight cycles).

The **Graham Scan Algorithm** is used to find the convex hull of a set of points. The convex hull is the smallest convex polygon that contains all the points in a 2D plane.

Lab Questions:

1. Implement Floyd-Warshall's Algorithm for all-pairs shortest path

Code:

```
#include <iostream>
#include <vector>
#include <limits>
using namespace std;

void FloydWarshall(int V, vector<vector<int>>& graph) {

    vector<vector<int>> dist = graph;

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] != INT_MAX && dist[k][j] != INT_MAX &&
                    dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

```

    }
}

cout << "Shortest distances between every pair of vertices:\n";
for (int i = 0; i < V; i++) {
    for (int j = 0; j < V; j++) {
        if (dist[i][j] == INT_MAX)
            cout << "INF ";
        else
            cout << dist[i][j] << " ";
    }
    cout << endl;
}
}

int main() {
    int V = 4; // Number of vertices
    vector<vector<int>> graph(V, vector<int>(V, INT_MAX));

    graph[0][0] = 0; graph[0][1] = 3; graph[0][2] = 5; graph[0][3] = INT_MAX;
    graph[1][0] = INT_MAX; graph[1][1] = 0; graph[1][2] = 1; graph[1][3] = 9;
    graph[2][0] = INT_MAX; graph[2][1] = INT_MAX; graph[2][2] = 0; graph[2][3] = 2;
    graph[3][0] = INT_MAX; graph[3][1] = INT_MAX; graph[3][2] = INT_MAX; graph[3][3]
= 0;

    FloydWarshall(V, graph);

    return 0;
}

```

Output:

```

Shortest distances between every pair of vertices:
0 3 4 5
INF 0 1 3
INF INF 0 2
INF INF INF 0

```

2. Implement Graham Scan Algorithm (Convex Hull)

Code:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Point {
    int x, y;
};

int orientation(Point p, Point q, Point r) {
    int val = (q.y - p.y) * (r.x - q.x) - (q.x - p.x) * (r.y - q.y);
    if (val == 0) return 0;
    return (val > 0) ? 1 : 2;
}

vector<Point> grahamScan(vector<Point>& points) {
    Point p0 = *min_element(points.begin(), points.end(), [](Point a, Point b) {
        return a.y < b.y || (a.y == b.y && a.x < b.x);
    });

    sort(points.begin(), points.end(), [p0](Point a, Point b) {
        int o = orientation(p0, a, b);
        if (o == 0) return (a.x - p0.x) * (a.x - p0.x) + (a.y - p0.y) * (a.y - p0.y)
        <
            (b.x - p0.x) * (b.x - p0.x) + (b.y - p0.y) * (b.y - p0.y);
        return o == 2;
    });

    vector<Point> hull;
    hull.push_back(p0);

    for (const Point& p : points) {
        while (hull.size() >= 2 && orientation(hull[hull.size() - 2],
hull[hull.size() - 1], p) != 2) {
            hull.pop_back();
        }
        hull.push_back(p);
    }

    return hull;
}

int main() {

```



```
vector<Point> points = {{0, 3}, {2, 2}, {1, 1}, {2, 1}, {3, 0}, {0, 0}, {3, 3}};  
vector<Point> hull = grahamScan(points);  
  
for (const Point& p : hull) {  
    cout << "(" << p.x << ", " << p.y << ")\n";  
}  
  
return 0;  
}
```

Output:

Points in the convex hull are:

(0, 0)
(3, 0)
(3, 3)
(0, 3)

7. String Matching Algorithms:

Aim: Understand the theory behind String Matching Algorithms, Implement the Naïve String Matching Algorithm and Rabin-Karp String Matching Algorithm.

Introduction: String matching algorithms are fundamental in computer science for finding a pattern in a text. They are widely used in searching, data mining, bioinformatics, and many other fields. In this module, we will explore two classic string matching algorithms.

Lab Questions:

1. Implement Naïve String Matching Algorithm

Code:

```
#include <iostream>
#include <string>
using namespace std;

void NaiveStringMatching(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();

    for (int i = 0; i <= n - m; i++) {
        int j = 0;
        while (j < m && text[i + j] == pattern[j]) {
            j++;
        }
        if (j == m) {
            cout << "Pattern found at index " << i << endl;
        }
    }
}

int main() {
    string text = "ABABDABACDABABCABAB";
    string pattern = "ABAB";

    NaiveStringMatching(text, pattern);
}
```

```
    return 0;
}
```

Output:

```
Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
Pattern found at index 17
```

2. Implement Rabin-Karp Algorithm for String Matching

Code:

```
#include <iostream>
#include <string>
#include <cmath>
using namespace std;

const int d = 256;
const int q = 101;

void RabinKarp(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();
    int i, j;
    int patternHash = 0;
    int textHash = 0;
    int h = 1;

    // The value of h is "pow(d, m-1)%q"
    for (i = 0; i < m - 1; i++) {
        h = (h * d) % q;
    }

    for (i = 0; i < m; i++) {
        patternHash = (d * patternHash + pattern[i]) % q;
        textHash = (d * textHash + text[i]) % q;
    }

    // Slide the pattern over the text one by one
    for (i = 0; i <= n - m; i++) {
        // If the hash values match, check the characters one by one
```

```

    if (patternHash == textHash) {
        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j]) {
                break;
            }
        }
        if (j == m) {
            cout << "Pattern found at index " << i << endl;
        }
    }

    // Calculate the hash value for the next window of text
    if (i < n - m) {
        textHash = (d * (textHash - text[i] * h) + text[i + m]) % q;
        if (textHash < 0) {
            textHash = (textHash + q);
        }
    }
}

int main() {
    string text = "ABABDABACDABABCABAB";
    string pattern = "ABAB";

    RabinKarp(text, pattern);

    return 0;
}

```

Output:

```

Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
Pattern found at index 17

```

8. Miscellaneous

Aim: To learn about different Miscellaneous Algorithms & Problem Solving: Extended Euclid's Algorithm (GCD), Prime Number Generation Algorithms, N-Queen Problem, Backtracking Problems, Modular Exponentiation, Fractional Knapsack Problem.

Introduction:

This lab covers a variety of algorithms that are essential in solving problems encountered in competitive programming, optimization, and number theory. We will explore algorithms such as the Extended Euclid's Algorithm to compute the GCD (Greatest Common Divisor), Prime Number Generation, the N-Queen Problem using backtracking, Modular Exponentiation, and the Fractional Knapsack Problem, which is a classic greedy algorithm.

Lab Questions:

1. Compute the Greatest Common Divisor (GCD) using Extended Euclid's Algorithm.

Code:

```
#include <iostream>
using namespace std;

int extendedGCD(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int gcd = extendedGCD(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return gcd;
}

int main() {
    int a = 30, b = 12, x, y;
```

```

int gcd = extendedGCD(a, b, x, y);
cout << "GCD: " << gcd << "\n";
cout << "Coefficients: x = " << x << ", y = " << y << endl;
return 0;
}

```

Output:

```

GCD: 6
Coefficients: x = -1, y = 3

```

2. Prime Number Generation (Sieve of Eratosthenes)

Code:

```

#include <iostream>
#include <vector>
using namespace std;

void sieve(int n) {
    vector<bool> prime(n + 1, true);
    prime[0] = prime[1] = false;

    for (int i = 2; i * i <= n; i++) {
        if (prime[i]) {
            for (int j = i * i; j <= n; j += i) {
                prime[j] = false;
            }
        }
    }

    cout << "Primes up to " << n << ":\n";
    for (int i = 2; i <= n; i++) {
        if (prime[i]) {
            cout << i << " ";
        }
    }
    cout << endl;
}

int main() {
    int n = 50;
    sieve(n);
    return 0;
}

```

Output:

```
Primes up to 50:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

3. N-Queen Problem (Backtracking)

Code:

```
#include <iostream>
#include <vector>
using namespace std;

bool isSafe(int row, int col, vector<vector<int>>& board, int N) {
    for (int i = 0; i < row; i++) {
        if (board[i][col] == 1 || (col - (row - i) >= 0 && board[i][col - (row - i)]
== 1) || (col + (row - i) < N && board[i][col + (row - i)] == 1)) {
            return false;
        }
    }
    return true;
}

bool solveNQueen(vector<vector<int>>& board, int row, int N) {
    if (row == N) {
        for (auto& r : board) {
            for (auto& c : r) cout << c << " ";
            cout << endl;
        }
        return true;
    }

    for (int col = 0; col < N; col++) {
        if (isSafe(row, col, board, N)) {
            board[row][col] = 1;
            if (solveNQueen(board, row + 1, N)) return true;
            board[row][col] = 0; // Backtrack
        }
    }
    return false;
}

int main() {
    int N = 4;
```

```

vector<vector<int>> board(N, vector<int>(N, 0));
if (!solveNQueen(board, 0, N)) {
    cout << "Solution does not exist\n";
}
return 0;
}

```

Output:

```

1 0 0 0
0 0 1 0
0 0 0 1
0 1 0 0

```

4. Modular Exponentiation:

Code:

```

#include <iostream>
using namespace std;

int modularExponentiation(int base, int exp, int mod) {
    int result = 1;
    base = base % mod;

    while (exp > 0) {
        if (exp % 2 == 1) {
            result = (result * base) % mod;
        }
        exp = exp >> 1;
        base = (base * base) % mod;
    }
    return result;
}

int main() {
    int base = 5, exp = 3, mod = 13;
    cout << "Result: " << modularExponentiation(base, exp, mod) << endl;
    return 0;
}

```

Output:

```

Result: 8

```


5. Fractional Knapsack Problem

Code:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

struct Item {
    int value;
    int weight;
};

bool compare(Item a, Item b) {
    return (double)a.value / a.weight > (double)b.value / b.weight;
}

double fractionalKnapsack(int W, vector<Item>& items, int n) {
    sort(items.begin(), items.end(), compare);
    double totalValue = 0.0;

    for (int i = 0; i < n; i++) {
        if (W == 0) break;

        if (items[i].weight <= W) {
            W -= items[i].weight;
            totalValue += items[i].value;
        } else {
            totalValue += items[i].value * ((double)W / items[i].weight);
            break;
        }
    }
    return totalValue;
}

int main() {
    int W = 50;
    vector<Item> items = {{60, 10}, {100, 20}, {120, 30}};
    int n = items.size();

    cout << "Maximum value in Knapsack: " << fractionalKnapsack(W, items, n) << endl;
    return 0;
}

```

Output:

Maximum value in Knapsack: 240